CS 486/686
Dissecting Diffusion and World Models

Yuntian Deng

Lecture 24

From noise to images to interactive worlds

Learning goals

- Explain diffusion as **iterative denoising**.
- Trace a **text-to-image** pipeline.
- Explain **latent diffusion** and **guidance**.
- Connect diffusion to **next-frame world models**.

From understanding to generation

L23

image + text → answer

L24

text / action / context → image or frame

The big question

a watercolor painting of a robot teaching CS486

How can a model turn this sentence into pixels?

Why not generate pixels directly?

huge

many pixel values

ambiguous

many valid images

structured

global and local details

Diffusion in one picture

Start from random noise and repeatedly denoise.

Training runs the movie backward

Create noisy examples from real images.

The forward noising process

$$x_t = \sqrt{\alpha_t}\, x_0 + \sqrt{1 - \alpha_t}\, \epsilon$$

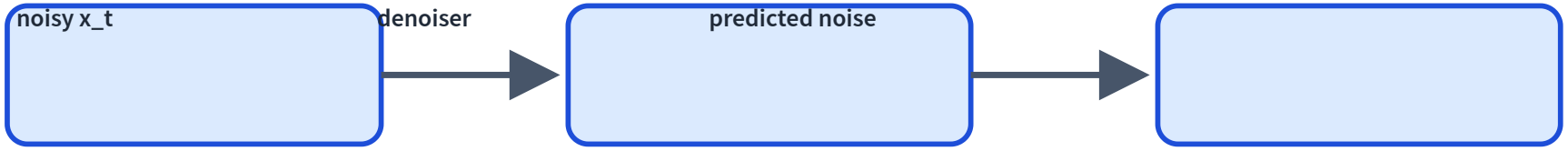
x_0: clean image

ϵ: Gaussian noise

t: noise level

x_t: noisy image

The denoiser's job



Input: noisy image, timestep, optional condition. Output: noise estimate.

Noise-prediction loss

$$L = \|\epsilon - \hat{\epsilon}_\theta(x_t, t, c)\|^2$$

Diffusion training is supervised learning because we know the noise we added.

Reverse process at inference

Apply the denoiser repeatedly from pure noise.

The sampling loop

```
x = randn(shape)
for t in scheduler.timesteps:
    eps = denoiser(x, t, condition)
    x = scheduler.step(x, eps, t)
image = decode(x)
```

The whole generator is an iterative loop.

The scheduler decides the steps

many steps

slower, often better

turbo steps

faster, distilled behavior

The denoiser predicts direction; the scheduler decides how to move.

Text-to-image pipeline

Text conditions every denoising step.

Where does text enter?

The denoiser attends to text embeddings while cleaning the image latent.

image latent

what is being cleaned

text embedding

what it should become

Guidance is a prompt-adherence dial

low guidance

diverse, may ignore prompt

high guidance

faithful, may distort

Classifier-free guidance

$$\hat{\epsilon} = \epsilon_{\text{uncond}} + s(\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$$

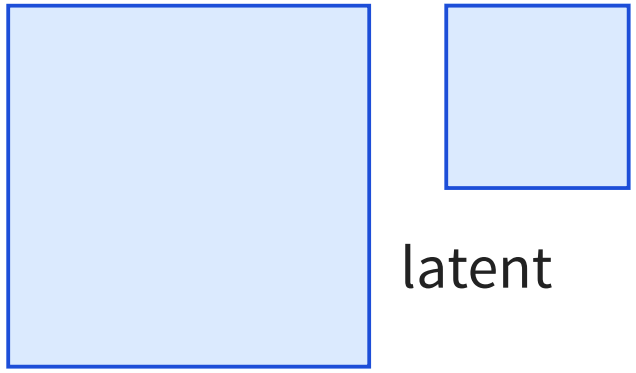
ϵ_{uncond}: no prompt

ϵ_{cond}: with prompt

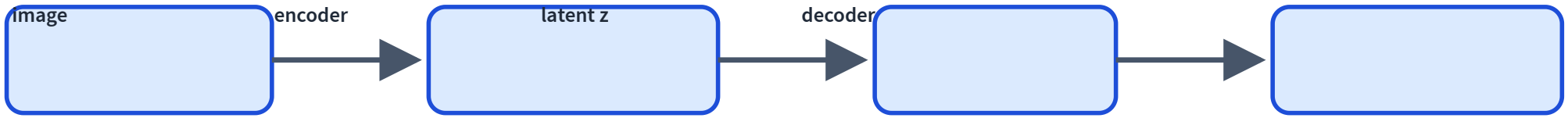
s: guidance scale

Amplify the prompt direction.

Pixel space is expensive



Autoencoders return



The compressed latent space is where Stable Diffusion works.

Latent diffusion

Denoise a compressed representation, then decode to an image.

Why latent diffusion matters

cheaper

less compute

smaller

smaller tensors

practical

higher resolution

Stable-Diffusion-style components

text encoder

prompt embedding

denoiser

predict noise

scheduler

step rule

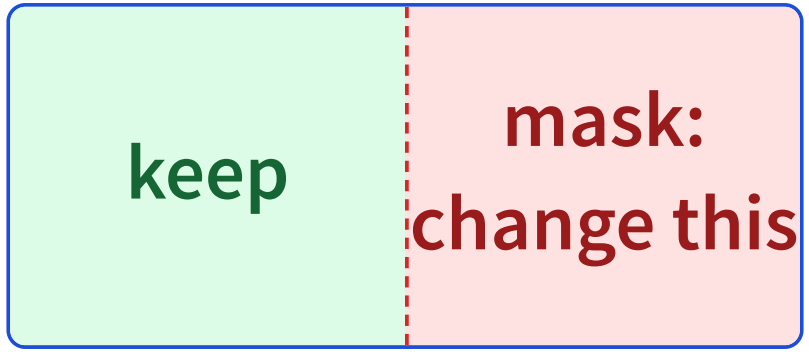
VAE decoder

latent to pixels

Trace: text to image

Prompt: `a blue robot holding a cup`

Image editing is constrained generation



Keep known pixels fixed; denoise only the missing or masked region.

Text is not the only control signal

sketch

rough shape

edges

structure

depth

geometry

pose

body layout

Failure mode: prompt mismatch

Prompt asks for **three cups**; image shows two.

Prompts are strong hints, not hard constraints.

Failure mode: text and fine details

Diffusion often struggles with exact words, symbols, fingers, and small objects.

Local detail requires global consistency.

Failure mode: artifacts and bias

artifacts

distorted objects

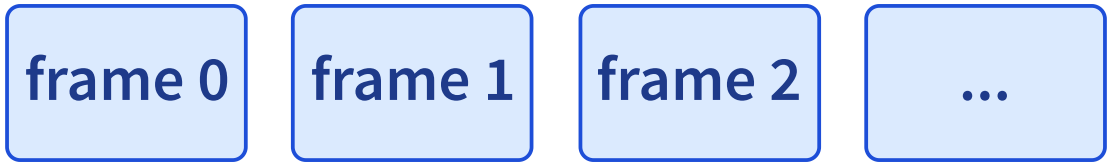
bias

training data patterns

safety

unsafe generation

From images to video



Video generation adds temporal consistency.

Next-frame prediction

$$P(\text{frame}_t \mid \text{previous frames},\ \text{actions})$$

Like next-token prediction, but outputs are frames or latents.

Actions matter

previous screen + mouse click → next screen

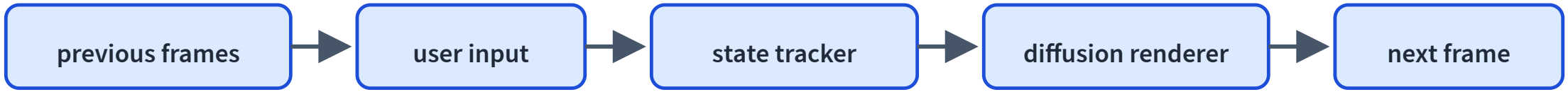
An interactive world model must condition on what the user does.

NeuralOS motivation

Instead of manually programming every UI transition, learn to predict the next screen from prior screens and user inputs.

A UI becomes a generative world model.

NeuralOS-style pipeline



NeuralOS as course capstone

state

hidden UI state

actions

mouse/keyboard

prediction

next frame

generation

diffusion renderer

World models can drift

temporal drift

state changes incorrectly

compounding errors

mistakes feed back

wrong actions

input misinterpreted

Optional browser demo path

```
const pipeline = new SDPipeline({
  model: "sd-turbo",
  provider: "webgpu",
});
const image = await pipeline.generate({ prompt, steps: 1 });
```

Small distilled diffusion models can run locally, but hardware support varies.

What did we dissect?

L21

text LM inference

L23

VLM understanding

L24

diffusion/world generation

Modern AI landscape

LLMs

text/code generation

VLMs

image understanding

Diffusion

image/video generation

World models

interactive prediction

Agents

models in action loops

Systems

retrieval, tools, safety

What this does not cover

math depth

score matching / SDEs

production policy

safety and deployment

full video training

large-scale recipes

all details

working mental model first

Recap

✓ Diffusion learns to predict noise.

✓ Sampling starts from noise and denoises iteratively.

✓ Text conditions denoising through embeddings/cross-attention.

✓ Guidance controls prompt adherence.

✓ Latent diffusion denoises compressed representations.

✓ World models predict future frames conditioned on actions.